
1. Code -(Attitude detection using Data Acquisition)

```
function attitude_data(varargin)
% (DATA RETRIEVAL - AIRPLANE ATTITUDE)
% Implements: IMU acquisition -> preprocessing -> sensor fusion -> MATLAB
workspace -> visualization
% Matches the PDF flow: Sensors(IMU) -> Acquire -> Filter -> Compute
Orientation -> Workspace -> Visualization.
% Now supports name-value inputs to change gains/filters/sim parameters WITHOUT
touching the pipeline.
%
% Examples:
%   attitude_data('ALPHA_ACC',0.08,'ALPHA_MAG',0.02,'SIM_TIME',60)
%   attitude_data('MODE','SERIAL','SERIAL_PORT',"COM7",'BAUD',921600)
%
attitude_data('SIM_roll_amp_deg',20,'SIM_pitch_amp_deg',12,'SIM_yaw_rate_dps',8
)
%
% Optional interactive prompting:
%   % Uncomment the call to promptForParams(P) below to pop a dialog when
nargin==0
%% ===== INPUTS / DEFAULTS =====
P = struct( ...
    'MODE','SIMULATE', ...           % 'SIMULATE' | 'SERIAL'
    'SIM_TIME',10, ...               % seconds
    'FS',100, ...                    % sample rate Hz
    'SERIAL_PORT',"COM5", ...        % serial port
    'BAUD',115200, ...               % serial baud
    ... % (line continuation ok here)
    'ALPHA_ACC',0.02, ...            % accel tilt fusion gain (0..1)
    'ALPHA_MAG',0.02, ...            % mag heading fusion gain (0..1)
    ... % (line continuation ok here)
    'LPF_CUTOFF_ACCEL',5, ...        % Hz
    'LPF_CUTOFF_MAG',5, ...          % Hz
    'LPF_CUTOFF_GYRO',25, ...        % Hz
    ... % (line continuation ok here)
    'gyro_bias',[0 0 0], ...         % rad/s (if known)
    'yaw_bias',0, ...                % rad
    'mag_declination_deg',0, ...      % degrees (converted to rad later)
    ... % (line continuation ok here)
    'SIM_roll_amp_deg',10, ...        % deg
    'SIM_pitch_amp_deg',6, ...        % deg
    'SIM_yaw_rate_dps',10/5 ...       % deg/s (same as prior 10 deg / 5 s)
);
```

```

P = parseNV(P, varargin{:});
% OPTIONAL interactive prompts when no args:
% if nargin==0
%     P = promptForParams(P);
% end
%% ===== DERIVED / SETUP =====
MODE          = upper(string(P.MODE));
SIM_TIME      = P.SIM_TIME;
FS            = P.FS;
SERIAL_PORT   = P.SERIAL_PORT;
BAUD          = P.BAUD;
ALPHA_ACC     = P.ALPHA_ACC;
ALPHA_MAG     = P.ALPHA_MAG;
LPF_CUTOFF_ACCEL = P.LPF_CUTOFF_ACCEL;
LPF_CUTOFF_MAG  = P.LPF_CUTOFF_MAG;
LPF_CUTOFF_GYRO = P.LPF_CUTOFF_GYRO;
gyro_bias     = P.gyro_bias;
yaw_bias      = P.yaw_bias;
mag_declination = deg2rad(P.mag_declination_deg);
% Simulator configuration
SIM.roll_amp  = deg2rad(P.SIM_roll_amp_deg);
SIM.pitch_amp = deg2rad(P.SIM_pitch_amp_deg);
SIM.yaw_rate  = deg2rad(P.SIM_yaw_rate_dps);
%% ===== CONSTANTS / PREALLOC =====
dt = 1/FS;
N = max(1, round(SIM_TIME*FS));
g = 9.80665;
deg = @(x) x*180/pi;
% Pre-allocate logs
tlog = zeros(N,1);
alog = zeros(N,3);
glog = zeros(N,3);
mlog = zeros(N,3);
rpy = zeros(N,3); % [roll pitch yaw] in rad
% Simple digital one-pole low-pass filters
accellPF = onePoleLPF(LPF_CUTOFF_ACCEL, FS);
gyroLPF  = onePoleLPF(LPF_CUTOFF_GYRO,  FS);
magLPF   = onePoleLPF(LPF_CUTOFF_MAG,   FS);
% Figure for visualization
[fig, axRPY, axHorizon] = makeFigure(); %#ok<ASGLU>
% Initial attitude estimate from gravity + magnetic field
init_est = true;
%% ===== ACQUISITION LOOP =====
if strcmpi(MODE, 'SERIAL')
    s = serialport(SERIAL_PORT, BAUD, "Timeout", 1);
    configureTerminator(s, "LF");
    disp("Reading IMU from " + SERIAL_PORT + " @ " + BAUD + " baud...");
else
    disp("SIMULATING IMU data @ " + FS + " Hz for " + SIM_TIME + " s...");

```

```

end
roll=0; pitch=0; yaw=0; % current attitude (rad)
tic;
for k = 1:N
    tlog(k) = (k-1)*dt;
    % ----- Acquire Raw Sensor Data -----
    if strcmpi(MODE, 'SERIAL')
        line = safeReadLine(s);
        if isempty(line); continue; end
        vals = sscanf(line, '%f,%f,%f,%f,%f,%f,%f,%f,%f,%f');
        if numel(vals) ~= 9
            % Try space-delimited fallback
            vals = sscanf(line, '%f %f %f %f %f %f %f %f %f %f');
            if numel(vals) ~= 9
                continue; % skip malformed line
            end
        end
        ax=vals(1); ay=vals(2); az=vals(3); % m/s^2
        gx=vals(4); gy=vals(5); gz=vals(6); % rad/s
        mx=vals(7); my=vals(8); mz=vals(9); % uT
    else
        % ----- SIMULATED IMU -----
        [ax,ay,az,gx,gy,gz,mx,my,mz] = simulateIMU_param(tlog(k), g, SIM);
    end
    % Log raw
    alog(k,:) = [ax ay az];
    glog(k,:) = [gx gy gz];
    mlog(k,:) = [mx my mz];
    % ----- Preprocessing (LPF + bias) -----
    acc_f = accelLPF([ax ay az]);
    gyr_f = gyroLPF([gx gy gz] - gyro_bias);
    mag_f = magLPF([mx my mz]);
    % ----- Sensor Fusion (Complementary) -----
    if init_est
        [r0,p0] = tiltFromAccel(acc_f, g);
        y0 = yawFromMag(mag_f, r0, p0) + mag_declination - yaw_bias;
        roll=r0; pitch=p0; yaw=y0;
        init_est=false;
    else
        % Predict via gyro
        [roll,pitch,yaw] = integrateEuler(roll,pitch,yaw, gyr_f, dt);
        % Correct tilt using accel
        [r_acc,p_acc] = tiltFromAccel(acc_f, g);
        roll = wrapAngle( (1-ALPHA_ACC)*roll + ALPHA_ACC*r_acc );
        pitch = wrapAngle( (1-ALPHA_ACC)*pitch + ALPHA_ACC*p_acc );
        % Correct heading using mag
        y_mag = yawFromMag(mag_f, roll, pitch) + mag_declination - yaw_bias;
        % use shortest-angle blend
        dy = wrapAngle(y_mag - yaw);
    end
end

```

```

        yaw = wrapAngle( yaw + ALPHA_MAG*dy );
    end
    rpy(k,:) = [roll pitch yaw];
    % ----- Visualization -----
    if mod(k, max(1, round(FS/30))) == 0 || k==1
        updateRPYPlot(axRPY, tlog(1:k), deg(rpy(1:k,:)));
        updateHorizon(axHorizon, roll, pitch);
        drawnow limitrate;
    end
end
% Clean up serial
if strcmpi(MODE,'SERIAL')
    try, clear s; catch, end %#ok<TRYNC>
end
%% ===== EXPORT TO WORKSPACE =====
AttitudeLog = table( ...
    tlog, ...
    alog(:,1), alog(:,2), alog(:,3), ...
    glog(:,1), glog(:,2), glog(:,3), ...
    mlog(:,1), mlog(:,2), mlog(:,3), ...
    rpy(:,1), rpy(:,2), rpy(:,3), ...
    'VariableNames',
    {'t_s','ax','ay','az','gx','gy','gz','mx','my','mz','roll_rad','pitch_rad','yaw_
    _rad'});
assignin('base','AttitudeLog',AttitudeLog);
disp('Exported AttitudeLog (table) to MATLAB workspace.');
```

% Quick summary table (last sample + mean of last 10%)

```

LastN = max(1, round(0.1*height(AttitudeLog)));
Summary = table( ...
    AttitudeLog.t_s(end), ...
    rad2deg(AttitudeLog.roll_rad(end)), ...
    rad2deg(AttitudeLog.pitch_rad(end)), ...
    rad2deg(AttitudeLog.yaw_rad(end)), ...
    rad2deg(mean(AttitudeLog.roll_rad(end-LastN+1:end))), ...
    rad2deg(mean(AttitudeLog.pitch_rad(end-LastN+1:end))), ...
    rad2deg(mean(AttitudeLog.yaw_rad(end-LastN+1:end))), ...
    'VariableNames', {'t_end_s','roll_end_deg','pitch_end_deg','yaw_end_deg',
    ...
    'roll_mean_last10pct_deg','pitch_mean_last10pct_deg','yaw_mean_last10pct_deg'})
;
disp(' ');
disp('===== SUMMARY (Attitude) =====');
disp(Summary);
%% ===== END MAIN =====
end
%% ----- Helper Functions -----
function filt = onePoleLPF(fc, fs)
    % Simple 1st order LPF (stateful). If fc<=0, acts as pass-through.
```

```

if fc <= 0
    alpha = 0; % pass-through
else
    alpha = exp(-2*pi*fc/fs);
end
state = [0 0 0];
first = true;
function y = step(x)
    if first
        state = x;
        first = false;
    elseif alpha == 0
        % pass-through (no filtering)
        state = x;
    else
        state = alpha*state + (1-alpha)*x;
    end
    y = state;
end
filt = @step;
end
function [roll,pitch,yaw] = integrateEuler(roll,pitch,yaw, gyro, dt)
    % gyro = [p q r] in rad/s, 321 (yaw-pitch-roll) Euler rates mapping
    p = gyro(1); q = gyro(2); r = gyro(3);
    % Euler kinematics matrix
    T = [1, sin(roll)*tan(pitch), cos(roll)*tan(pitch);
         0, cos(roll), -sin(roll);
         0, sin(roll)/cos(pitch), cos(roll)/cos(pitch)];
    eul_dot = T*[p; q; r];
    roll = wrapAngle(roll + eul_dot(1)*dt);
    pitch = wrapAngle(pitch + eul_dot(2)*dt);
    yaw = wrapAngle(yaw + eul_dot(3)*dt);
end
function [roll_acc,pitch_acc] = tiltFromAccel(a, g)
    % a = [ax ay az] m/s^2. Use gravity direction for tilt.
    ax=a(1); ay=a(2); az=a(3);
    % clamp to avoid invalid domain
    ax = max(min(ax/g, 1), -1);
    ay = max(min(ay/g, 1), -1);
    az = max(min(az/g, 1), -1);
    pitch_acc = atan2(-ax, sqrt(ay^2 + az^2)); % nose up +ve
    roll_acc = atan2(ay, az); % right wing down +ve
end
function yaw_mag = yawFromMag(m, roll, pitch)
    % Tilt-compensated heading from magnetometer (uT)
    mx=m(1); my=m(2); mz=m(3);
    % Rotate mag into horizontal plane using current roll/pitch
    cr = cos(roll); sr = sin(roll);
    cp = cos(pitch); sp = sin(pitch);

```

```

mx_h = mx*cp + mz*sp;
my_h = mx*sr*sp + my*cr - mz*sr*cp;
yaw_mag = atan2(-my_h, mx_h); % ENU convention (edit if needed)
yaw_mag = wrapAngle(yaw_mag);
end
function a = wrapAngle(a)
    a = atan2(sin(a), cos(a));
end
function line = safeReadLine(s)
    try
        line = readline(s);
        line = strip(line);
    catch
        line = "";
    end
end
function [ax,ay,az,gx,gy,gz,mx,my,mz] = simulateIMU_param(t, g, SIM)
    % Parameterized simulator: change roll/pitch amplitudes and yaw rate to
    % get different waveform shapes for testing.
    roll_amp = SIM.roll_amp; % rad
    pitch_amp = SIM.pitch_amp; % rad
    yaw_rate = SIM.yaw_rate; % rad/s
    roll = roll_amp * sin(2*pi*0.25*t);
    pitch = pitch_amp * sin(2*pi*0.20*t + 0.7);
    yaw = yaw_rate * t;
    % Angular rates (approx derivatives)
    p = 2*pi*0.25*roll_amp * cos(2*pi*0.25*t);
    q = 2*pi*0.20*pitch_amp * cos(2*pi*0.20*t + 0.7);
    r = yaw_rate;
    % Gravity projection (no linear accel)
    ax = -g*sin(pitch);
    ay = g*sin(roll).*cos(pitch);
    az = g*cos(roll).*cos(pitch);
    % Add small noise
    ax=ax + 0.05*randn; ay=ay + 0.05*randn; az=az + 0.05*randn;
    % Gyro (rad/s) + noise
    gx=p + 0.002*randn; gy=q + 0.002*randn; gz=r + 0.002*randn;
    % Magnetic field (uT)
    B = 48; % uT
    mx = B*cos(yaw) + 0.5*randn;
    my = B*sin(yaw) + 0.5*randn;
    mz = 5 + 0.5*randn; % small vertical component
end
function [fig, axRPY, axH] = makeFigure()
    fig = figure('Name','Flowchart-1: Attitude Data &
Visualization','Color','w');
    tiledlayout(2,1,'Padding','compact','TileSpacing','compact');
    axRPY = nexttile(1);
    hold(axRPY,'on'); grid(axRPY,'on');

```

```

    title(axRPY, 'Roll / Pitch / Yaw (deg)'); xlabel(axRPY, 'Time (s)');
    ylabel(axRPY, 'Angle (deg)');
    legend(axRPY,
    'Roll', 'Pitch', 'Yaw', 'Location', 'southoutside', 'Orientation', 'horizontal');
    axH = nexttile(2);
    axis(axH, [-1 1 -0.3 0.7]); axis(axH, 'equal'); grid(axH, 'on'); box(axH, 'on');
    title(axH, 'Artificial Horizon'); xlabel(axH, ''); ylabel(axH, '');
end
function updateRPYPlot(ax, t, rpy_deg)
    persistent h1 h2 h3
    if isempty(h1) || ~isvalid(h1)
        h1 = plot(ax, t, rpy_deg(:,1), 'LineWidth', 1.2);
        h2 = plot(ax, t, rpy_deg(:,2), 'LineWidth', 1.2);
        h3 = plot(ax, t, rpy_deg(:,3), 'LineWidth', 1.2);
    else
        set(h1, 'XData', t, 'YData', rpy_deg(:,1));
        set(h2, 'XData', t, 'YData', rpy_deg(:,2));
        set(h3, 'XData', t, 'YData', rpy_deg(:,3));
    end
    xlim(ax, [max(0, t(end)-10) t(end)+eps]); % sliding window
end
function updateHorizon(ax, roll, pitch)
    % Simple artificial horizon: sky (top) and ground (bottom) separated by a
    line
    cla(ax);
    % Horizon tilt is -roll; vertical shift ~ pitch
    R = [cos(-roll) -sin(-roll); sin(-roll) cos(-roll)];
    yShift = pitch/ (pi/6); % scale ~ +/-30deg
    pts = R * [-2 2; 0 0]; % horizon line
    % Draw sky and ground as patches
    skyX = [-2 2 2 -2];
    skyY = [ 0 0 2 2] + yShift;
    grdX = [-2 2 2 -2];
    grdY = [ 0 0 -2 -2] + yShift;
    patch(ax, skyX, skyY, [0.75 0.9 1.0], 'EdgeColor', 'none'); % sky
    patch(ax, grdX, grdY, [0.9 0.8 0.6], 'EdgeColor', 'none'); % ground
    hold(ax, 'on');
    plot(ax, pts(1,:), pts(2,:)+yShift, 'k', 'LineWidth', 2); % horizon line
    plot(ax, [0 0], [-0.1 0.1], 'k', 'LineWidth', 2); % center marker
    axis(ax, [-1 1 -0.3 0.7]); axis(ax, 'off');
end
function P = parseNV(P, varargin)
    % Simple name-value parser with basic validation
    if mod(numel(varargin),2)~=0
        error('Inputs must be name-value pairs.');
```

```
val = varargin{i+1};  
if ~ischar(name) && ~isstring(name)  
    error('Parameter names must be char or string.');end  
name = string(name);  
match = strcmpi(name, validNames);  
if ~any(match)  
    error('Unknown parameter name: %s', name);  
end  
P.(validNames{match}) = val;  
end  
end
```